

DESIGN AND ANALYSIS OF ALGORITHM (CS-327)

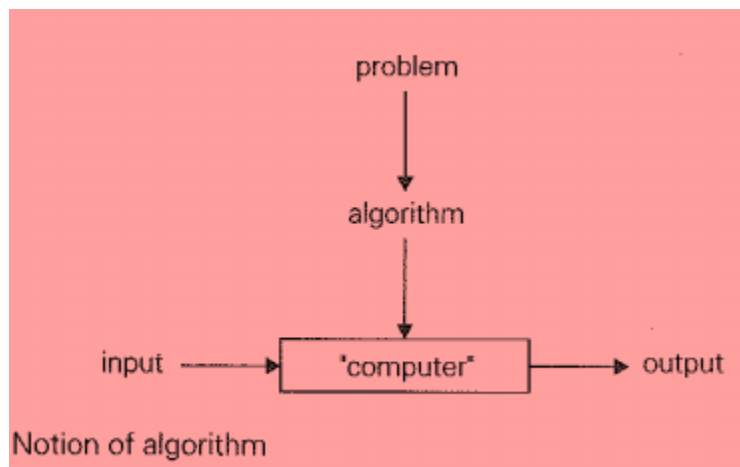
LECTURE 1

What is Algorithm

Finite sequence of set of physical possible steps, performed in correct possible order for solving a computational problem. It takes you from one consistent state to another consistent state whether both of these states may or may not be same. Algorithm must be language independent.

OR

An algorithm is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.



Properties of Algorithm :-

- It should be finite.
- It should produce at least one output.
- It should take zero or more input.
- It must be deterministic and give the same output for same input.
- Every step must do some work means every step should be effective.

Types of Algorithm :-

- Brute Force Algorithms.
- Recursive Algorithms.
- Backtracking Algorithms.
- Searching Algorithms.
- Sorting Algorithms.
- Divide and conquer Algorithms.
- Hashing Algorithms.
- Dynamic Programming Algorithms.
- Greedy Algorithms.
- Randomized Algorithm

Example :-

Problem1 -----solution1-----Algorithm1-----Machine1-----Time1

Problem2 -----solution2-----Algorithm2-----Machine2-----Time2

Scenarios :-

T1 < T2 ----- why ----- (Machine M1 is better)

T1 < T2 ----- why ----- (Algo1 is better)

T1 > T2 ----- why ----- (Algo2 is better)

Note :-

So with the Above Example we came to know , while comparing two different algorithms we make sure problem and machine must be constant.

Example of Algorithm :-

<u>Problem</u>	<u>Algorithm (Step by step method)</u>	<u>Results</u>
<u>Making a cup of tea</u>	1) Take an empty cup 2) Add hot water 3) Add tea 4) Add sugar 5) Add milk 6) Mix / Stir	<u>Cup of tea is ready</u>

Algorithms Notations :-

- Algorithm Name (Should be in capital form)
- Parameters (should be enclosed in parenthesis)
- Introductory comment (description and purpose of algorithm)
- Steps(finite steps)
- Comments (Each step starts with a comments, enclose in [], defines the purpose of step
- Input statement
- Output statement

Example of Algorithm :-

Algorithm (Step by step method)

Algorithm SUM (No1, No2, Result)

{ This Algorithm is used to read two numbers and print its sum }

Step1 [Read numbers No1 and No2]

Read (No1, No2)

Step2 [Calculate Sum]

Result = No1 + No2

Step3[Display result]

Write (result)

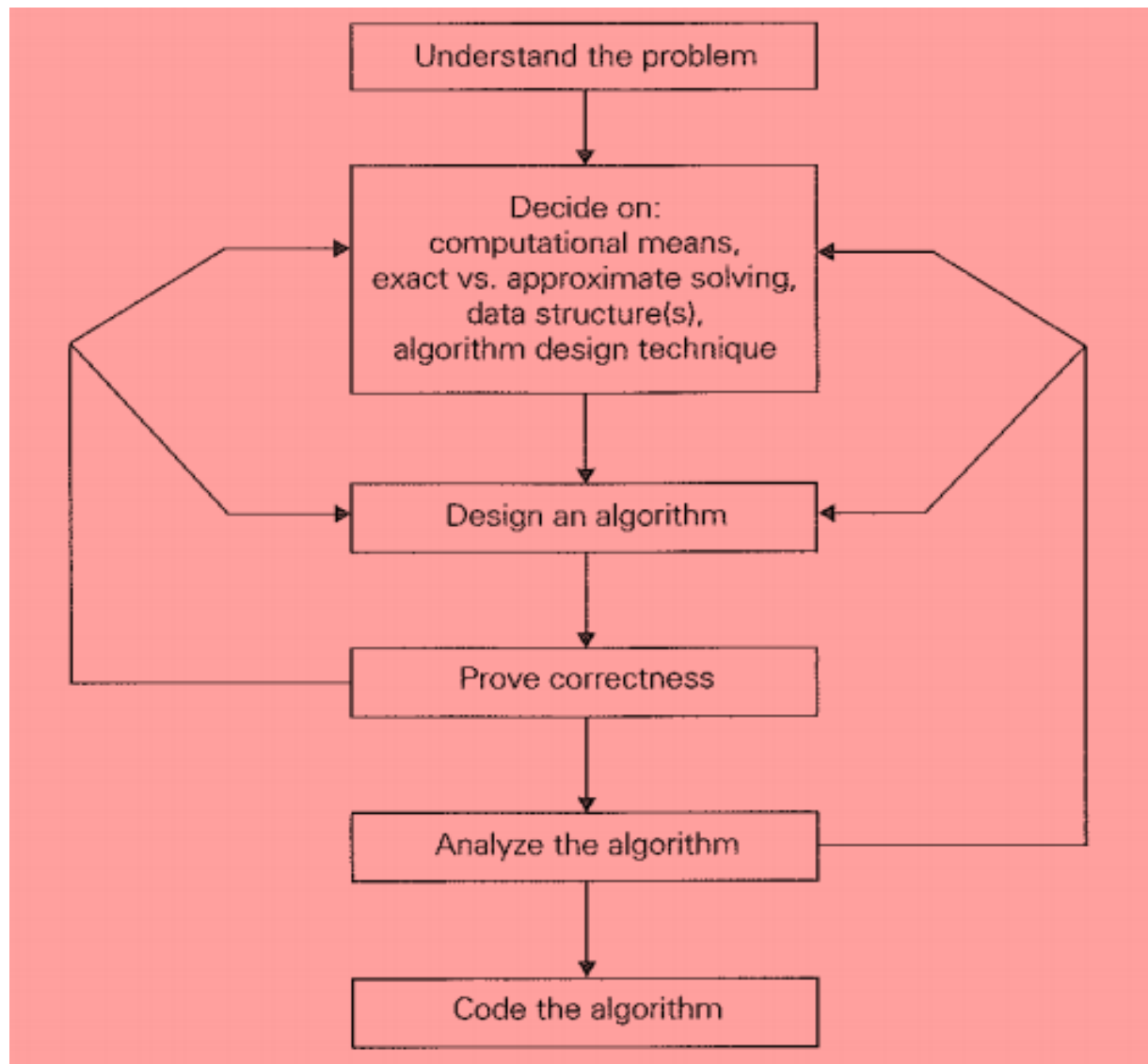
Step4 [Finish]

Exit

Fundamentals of Algorithmic Problem Solving :-

OR

How to design an algorithm :-



Algorithm Design and Analysis Process

Understanding the Problem

From a practical perspective, the first thing you need to do before designing an algorithm is to understand completely the problem given. Read the problem's description carefully and ask questions if you have any doubts about the problem.

Ascertaining the Capabilities of a Computational Device

Once you completely understand a problem, you need to ascertain the capabilities of the computational device the algorithm is intended for.

Algorithm Design Techniques

An algorithm design technique (or "strategy" or "paradigm") is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.

Proving an Algorithm's Correctness

Once an algorithm has been specified, you have to prove its correctness. That is, you have to prove that the algorithm yields a required result for every legitimate input in a finite amount of time. For some algorithms, proof of correctness is quite easy; for others, it can be quite complex. A common technique for proving correctness is to use mathematical induction because an algorithm's iterations provide a natural sequence of steps needed for such proofs.

Analyzing an Algorithm

There are two kinds of algorithm efficiency: time efficiency and space efficiency.

1. **Time** efficiency indicates how fast the algorithm runs.
2. **Space** efficiency indicates how much extra memory the algorithm needs.

Coding an Algorithm

So finally we code the above algorithm in your desired language.